# SpotLight: Title is TBD

Anonymous Author(s)
Submission Id: #xxx$_1$

## Abstract

## CCS Concepts

• **Computer systems organization** → **Cloud computing**; • **Software and its engineering** → *Scheduling*; *Software performance*; • **Computing methodologies** → Planning and scheduling.

## Keywords

xxx, yyy, zzz

## 1 Introduction

Hybrid spot/on-demand strategies [11, 34] have been extensively studied.

## 2 Motivation

Federated learning (FL) is now widely used for training machine learning models without centralizing training data [19]. As client populations grow, the infrastructure responsible for aggregating model updates must scale accordingly. Production FL deployments already coordinate tens of millions of devices, with system designers targeting deployments at the scale of billions of devices [3], placing significant demand on the aggregation layer. Existing frameworks such as Flower [2], IBM FL [18], FATE [17], and Papaya [8] are designed as single aggregator systems. In production FL deployments, the aggregation layer is responsible for managing the lifecycle of connecting clients, selecting and forwarding devices into training rounds, and combining their model updates into a global model. This breadth of responsibility places significant resource demands on a single aggregator, and as the number of clients grows, single aggregators become bottlenecks that increase the wall clock time of training, as clients are forced to wait longer for the aggregator to complete a round before receiving the next set of updates [10].

Papaya [8] and Flower [2] are both single-aggregator FL systems, and both carry fundamental limitations that follow directly from this design. FL payload sizes vary widely, from 22MB for a model with 11 million parameters [5] to over 140GB for large language models [21], and in a single-aggregator system every one of these updates must pass through and be processed by that one aggregator. This makes the aggregation step both network- and compute-intensive at scale, and as client load grows the aggregator becomes a bottleneck. In Flower, this bottleneck causes outright failures that stall training (Figure 1). Papaya's asynchronous execution model avoids this failure mode under the same workload, but the same server-side bottleneck still surfaces in a different form, as inconsistent client utilization, the fraction of a round a client spends training rather than waiting on an updated model (Figure 2).

A straightforward response to the limitations of a single aggregator is to add aggregators as client load increases. The stateful nature of FL aggregation, however, makes this non-trivial. Horizontally scaling the aggregation layer introduces a set of problems absent from the single-aggregator case.

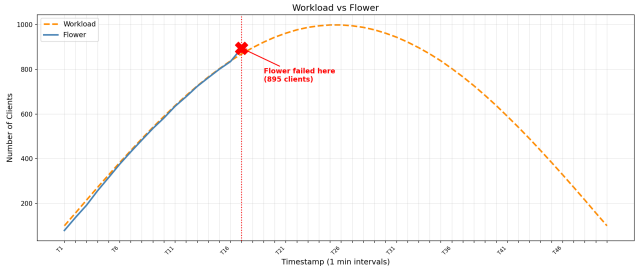



**Figure 1: Flower, configured with a 95% client quorum and 10MB payloads, causes training to stall at approximately 895 concurrent clients, where the rate of incoming updates exceeds the server's processing capacity.No on such failure was obeserved with Papaya for the same workload**

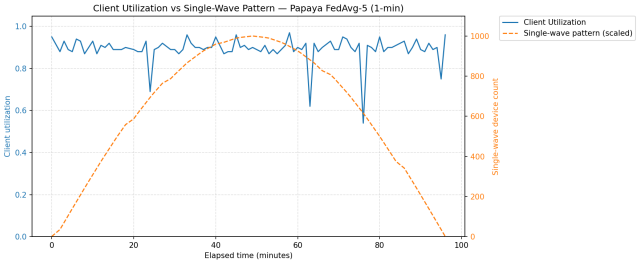



**Figure 2: Client utilization (fraction of round time spent on useful training work) for Papaya under a workload of up to 1,000 devices. Papaya completes the workload without failure, but utilization is inconsistent, dropping as low as 0.6–0.75 due to aggregator-side delay.**

The first such problem is maintaining global model consistency across aggregator instances. Each aggregator receives updates from only a subset of clients, producing a sub-global model accumulated from the subset of clients. The sub-global models produced by individual aggregators would require an additional step to produce a consistent global model that incorporates updates from all clients, a coordination requirement inherent to any system that partitions shared state across multiple nodes [15].

The second challenge with aggregator scaling is model staleness. Due to the differences in the subset of updates that aggregators individually receive, the model version a client trains on may diverge from the current global model. This divergence causes a subset of clients to take longer to converge as staleness increases [33]. The third challenge with aggregator scaling is scheduling client updates across aggregators, which is difficult for two reasons. First, aggregation is computationally expensive, so a scheduler that ignores load can bottleneck a single aggregator even while others sit idle.

**Design Insight 1:** Scaling FL aggregation requires more than replicating a single aggregator; consistency, staleness, scheduling, and state propagation costs multiply with each instance added. An explicit step is further required to merge sub-global models into a consistent global model. This motivates a two-tiered architecture in which coordination is bounded within each tier and a dedicated aggregation step merges sub-global models into a single global model.

Second, incoming client updates must be placed on an aggregator holding the current global model rather than an older one, to avoid wasting the client's training effort on a model the system has already moved past. The fourth challenge with aggregator scaling is propagating a common state across aggregators following aggregation. The updated global model, along with server-side optimizer state such as momentum or adaptive learning-rate terms maintained alongside the model, must be propagated to every aggregator [24]. The synchronization of the global model and server-side optimizer state must occur throughout the FL process, making it a persistent challenge. These four challenges illustrate that naively scaling single-aggregator systems horizontally in response to client load is insufficient for maintaining model consistency and client performance. The FL system architecture must therefore be redesigned from a single-aggregator model to one with FL-specific coordination, synchronization, and scheduling.

Aggregators in FL systems are deployed on always-on, on-demand cloud instances [10, 23].Since single aggregators are insufficient to handle large client populations, deployments must scale to multiple aggregators, driving up on-demand costs. Figure 3 projects the total aggregator cost for a 3-aggregator deployment training ResNet-18 on CIFAR-100, derived from our measured per-round aggregator time extrapolated to the 1,198 rounds reported for FedAvg convergence by Sun et al. [28]. Spot instances offer a cheaper alternative to on-demand instances [29, 32], being up to 88% cheaper [14]; however, spot instances can be preempted at any time by the cloud provider and come with no availability guarantees [9, 27]. Fault tolerance mechanisms are employed to preserve application state and enable recovery from such failures [4, 11, 25]. While techniques such as checkpointing and instance migration are commonly adopted for this purpose, each presents fundamental limitations when applied to FL aggregation systems. We identify three such limitations below. The first limitation is to do with Checkpointing, it is a commonly adopted FT technique that persists application state to durable storage, enabling recovery following a failure. In the context of FL aggregation, a naive checkpointing strategy saves the global model state after every aggregation update. However, as the number of aggregation updates grows, the cumulative I/O overhead of checkpointing every update grows proportionally. At scale, this overhead becomes a significant bottleneck, stalling the aggregator and degrading training throughput.

The second limitation is to do with migration,it is a FT technique that transfers the state of a failed instance to a newly provisioned replacement, allowing the system to resume operation. In the context of FL aggregation, this involves transferring the in-progress sub-global model, optimizer state, and client update buffer to a replacement aggregator following a spot preemption. However, cloud

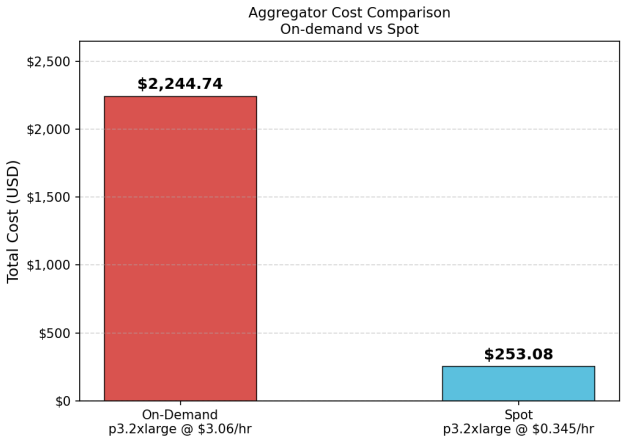



**Figure 3: Total aggregator cost for training ResNet-18 on CIFAR-100 to convergence (1,198 rounds [28]) across 3 aggregators on p3.2xlarge instances. On-demand pricing at $3.06/hr results in a total cost of $2,244.74, compared to $253.08 under spot pricing at $0.345/hr, an 88.7% reduction.**

**Design Insight 2:** Checkpointing, instance migration, and provider termination signals are each insufficient for FL aggregation on spot infrastructure. Effective fault tolerance requires decoupling aggregator state from instance lifecycle and acting on failure likelihood independently of provider-issued signals.

VM provisioning is not instantaneous; prior work has demonstrated that newly provisioned VMs incur substantial startup latency [7, 31]. During this provisioning window, the aggregator is unavailable, and for a system processing a large volume of client updates, this downtime results in lost updates, stalled training and increased time to reach target accuracy.

Finally Spot VM lifetimes are non-deterministic and have no prior knowledge of when preemption will occur. Although cloud providers issue a termination signal prior to reclamation, the notice period varies considerably across providers; AWS issues a two-minute warning [1] while GCP issues a 30-second warning [6]. The state transfer overhead inherent to FL aggregation systems may exceed the available notice window, rendering provider termination signals an insufficient basis for guaranteeing successful migration prior to instance reclamation.

These challenges collectively motivate a ground-up redesign of the FL aggregation architecture, one that natively supports horizontal scaling, tolerates spot instance preemption, and is built with cost efficiency in mind from the start. The next section presents SpotLight, a system designed around these requirements.

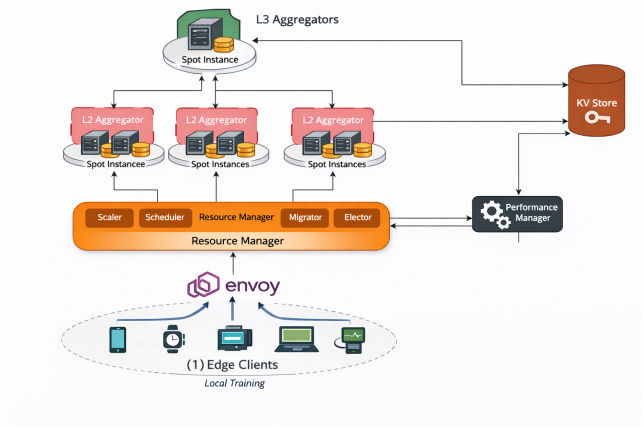



**Figure 4: Placeholder, will be replaced**

## 3 System Overview

SpotLight is a hierarchical, elastically scalable, fault-tolerant federated learning system that distributes aggregation and coordination responsibilities across independently scalable layers. The system comprises four components: the **Resource Manager (RM)**, **Sub-Global Aggregators**, **Global Aggregators**, and **Performance Manager**. Failover Edge Aggregators are designated standby nodes that extend the aggregation tier to the edge during periods of Sub-Global or Global Aggregator unavailability. The RM serves as the control plane, managing client membership, scheduling incoming client updates, and scaling both Sub-Global and Global Aggregators. The Sub-Global and Global Aggregators form the two-tiered aggregation plane: Sub-Global Aggregators accumulate updates from assigned clients and produce sub-global models, while Global Aggregators aggregate sub-global models into the global model. The Performance Manager observes load across the aggregation layers and exposes scaling signals; the RM consumes these signals to provision or deprovision Sub-Global and Global Aggregators horizontally. When aggregators become unavailable, SpotLight dynamically extends its aggregation hierarchy to the edge: Failover Edge Aggregators assume the role of the unavailable aggregator tier, and the RM retains and redelivers any in-flight updates to the next available aggregator, limiting disruption to training. All external client connections are fronted by Envoy, and MongoDB persists coordination state shared across the RM and aggregation layers. Section 4 describes each component in detail.

## 4 System Components

This section describes the key components of SpotLight in detail. Each component is designed with a focused responsibility, allowing individual parts of the system to scale and fail independently. We describe the Resource Manager, which serves as the control plane, followed by the Aggregators, which form the data plane and include Sub-Global Aggregators, Global Aggregators, and Failover Edge Aggregators.

### 4.1 Resource Manager

The Resource Manager (RM) serves as the control plane of SpotLight, maintaining a global view of system state and coordinating client placement, elastic scaling, and fault recovery across the aggregation tiers.The RM consists of five components: the Scheduler, the Scaler, the Performance Manager, the Leader Elector, and the Migrator. **Scheduler** is responsible for scheduling incoming client updates onto available Sub-Global Aggregators. The Scheduler maintains a live routing table of ready Sub-Global Aggregator pod addresses, built via periodic queries to the Kubernetes API; only pods in a ready state are admitted. Using this table, the Scheduler applies a load-aware placement policy, formally defined in Algorithm 1, to determine the target Sub-Global Aggregator for each incoming update. In asynchronous FL systems, client arrivals are bursty and unpredictable, meaning uniform distribution of updates across aggregators risks overloading individual nodes while others remain idle. To address this, the Scheduler prioritizes aggregators that are closest to triggering aggregation, maximizing batch efficiency and reducing client queue time. When multiple aggregators are equally close, the LP score, computed from queue length, active workers, and buffer state, is used as a tiebreaker to prefer less loaded aggregators. The Scheduler returns a primary and secondary placement target, where the secondary serves as a fallback in the event of transient connection failures.

**Scaler** is responsible for elastically scaling both Aggregator tiers in response to client load and system performance. Sub-Global Aggregator scaling is managed by the RM using the decision process defined in Algorithm 2, together with the client-based and utilization-based modules described in Algorithms 3 and 4. Global Aggregator scaling is managed by the RM using a memory utilization policy, provisioning replicas in response to observed memory pressure across the global aggregation tier.

Client-based scaling serves as the primary control policy for Sub-Global Aggregators. FL workload is fundamentally driven by the number of participating clients, so the system determines the required number of replicas using a configurable clients-per-replica threshold, ensuring proportional capacity growth as participation increases. However, provisioning the correct number of aggregators does not always guarantee performance. Bursty arrivals or heterogeneous aggregation costs can cause performance to degrade even when the replica count appears sufficient. A secondary utilization-based policy addresses this edge case by estimating effective workload using aggregation time and queue delay metrics, triggering scaling corrections when performance drops despite adequate provisioning. This corrective mechanism is evaluated only when the primary policy does not act, maintaining stability while preventing under-provisioning.

**Performance Manager** is responsible for dynamically adjusting the scaling parameters in response to observed system performance. The Performance Manager implements a Proportional-Integral-Derivative (PID) controller that monitors aggregation throughput and reacts to changes between consecutive control cycles. Unlike conventional PID controllers that track a fixed setpoint, the Performance Manager forgoes a fixed throughput target, as variable client participation and model complexity make it difficult to determine

**Algorithm 1** Load-Aware Placement

**Output:** $(primary\_addr,\ secondary\_addr)$
**Notation:**
$Q$ — total client updates queued at the aggregator
$\Theta$ — aggregation throughput in updates per second
$skip\_addrs$ — aggregator addresses excluded by the Migrator due to elevated preemption risk
$primary\_addr$ — highest priority placement target under normal operation
$secondary\_addr$ — fallback target if primary is unreachable; if both fail, Migrator escalates to Failover Edge Aggregators

routing_table ← GetLiveRoutingTable()
**if** routing_table is empty **then**
  **error** "No available aggregators"
candidates ← ∅
**for all** addr ∈ routing_table **do**
  **if** addr ∈ skip_addrs **then**
    **continue**
  pod ← ExtractPodName(addr)
  metrics ← GetMetricsForPod(pod)
  **if** metrics exist **then**
    $Q$ ← metrics.total_queue_length
    $\Theta$ ← metrics.aggregation_throughput
  **else**
    $Q \leftarrow 0$
    $\Theta \leftarrow 0$
  candidates ← candidates ∪{$(addr,\ Q,\ \Theta)$}
**if** candidates is empty **then**
  **error** "No available nodes after filtering"
Sort candidates in descending order of $Q$
Break ties using descending order of $\Theta$
primary ← candidates[0].addr
**if** $|candidates| \geq 2$ **then**
  secondary ← candidates[1].addr
**else**
  secondary ← None
**return** $(primary,\ secondary)$

an appropriate value without extensive tuning. Instead, the controller reacts to changes in throughput between cycles, tightening scaling parameters when throughput degrades and relaxing them when it improves. To prevent integral windup under a zero setpoint, the integral term applies exponential decay with factor $\gamma$ at each cycle and is clamped to $[-I_{max}, I_{max}]$, capturing sustained throughput degradation across consecutive cycles without diverging. The error signal $e(t)$, defined in Equation 1, captures this change between cycles. The controller output $u(t)$, defined in Equation 2, is mapped to scaling parameters across both aggregation tiers, forming a closed loop that allows each layer to adapt independently to variable client loads. The PID parameters used in the deployment are listed in Table 1.

$$e(t) = T_{\text{current}} - T_{\text{previous}} \tag{1}$$

**Algorithm 2** ScalingDecision

// **Collect Metrics**
$current\_clients$ ← QueryActiveClients()
$current\_replicas$ ← GetCurrentReplicas()
$current\_util$ ← ComputeClientUtilization()
$avg\_queue\_time$ ← GetAverageQueueTime()
$avg\_agg\_time$ ← GetAverageAggregationTime()
$scale\_threshold$ ← Config.SCALE_THRESHOLD
$target\_util$ ← Config.TARGET_UTILIZATION

// **1. Client-Based Policy**
$client\_decision$ ← ClientBasedScaling($current\_clients$, $current\_replicas$, $scale\_threshold$)
**if** $client\_decision \neq NONE$ **then**
  **return** $client\_decision$

// **2. Utilization-Based Policy**
$util\_decision$ ← UtilizationBasedScaling($current\_clients$, $avg\_agg\_time$, $avg\_queue\_time$, $current\_replicas$, $loop\_sleep\_time$, $target\_util$, $scale\_threshold$)
**if** $util\_decision \neq NONE$ **then**
  **return** $util\_decision$

**return** NONE

**Algorithm 3** ClientBasedScaling

**Input:** $num\_clients$, $current\_replicas$, $scale\_threshold$
**Output:** $desired\_replicas$ or NONE

$desired\_replicas \leftarrow \lceil num\_clients / scale\_threshold \rceil$
**if** $desired\_replicas > current\_replicas$ **then**
  **return** $desired\_replicas$ ▷ Scale Out
**else if** $desired\_replicas < current\_replicas$ **then**
  **return** $desired\_replicas$ ▷ Scale In
**else**
  **return** NONE

**Algorithm 4** UtilizationBasedScaling

**Input:** $num\_clients$, $avg\_agg\_time$, $avg\_queue\_time$, $current\_replicas$, $loop\_sleep\_time$, $target\_util$, $scale\_threshold$
**Output:** $desired\_replicas$ or NONE

$arrival\_rate \leftarrow num\_clients / loop\_sleep\_time$
$service\_time \leftarrow avg\_agg\_time + avg\_queue\_time$
$total\_workload \leftarrow arrival\_rate \times service\_time$
$capacity\_per\_replica \leftarrow scale\_threshold \times target\_util$
$desired\_replicas \leftarrow \lceil total\_workload / capacity\_per\_replica \rceil$
**if** $desired\_replicas > current\_replicas$ **then**
  **return** $desired\_replicas$ ▷ Scale Out
**else if** $desired\_replicas < current\_replicas$ **then**
  **return** $desired\_replicas$ ▷ Scale In
**else**
  **return** NONE

$$u(t) = K_p e(t) + K_i \cdot \text{clamp}(\gamma \cdot I_{t-1} + e(t) \cdot \Delta t,\ -I_{max},\ I_{max}) + K_d \frac{de(t)}{dt} \tag{2}$$

**Table 1: PID Controller Values**

| Parameter | Value |
|---|---|
| $K_p$ | 0.05 |
| $K_i$ | 0.02 |
| $K_d$ | 0.10 |
| $\gamma$ | 0.97 |
| $I_{max}$ | 10.0 |

**Leader Elector** ensures that at any given time, a single RM instance holds exclusive authority over scaling decisions, while follower instances handle scheduling using the load-aware placement algorithm. The RM scales horizontally in response to client load to avoid performance degradation; however, without a consensus mechanism this introduces the risk of split-brain scenarios where multiple instances make conflicting scaling decisions. The Leader Elector addresses this by using the Raft consensus protocol [22] to elect a single leader among RM instances. In addition to ensuring consistency in scaling decisions, this design also provides fault tolerance at the control plane. Upon failure of the leader, Raft automatically promotes a successor, ensuring failover does not interrupt scaling or scheduling operations.

**Migrator** proactively migrates workload away from Sub-Global Aggregator instances at risk of preemption. The Migrator continuously consumes the Kaplan-Meier failure likelihood signal (Section **??**) for each active Sub-Global Aggregator instance and, on detecting elevated preemption risk, notifies the Scheduler to exclude the at-risk pod from placement decisions. Incoming client updates are thereby redirected to healthy instances without interrupting training. In-buffer updates on the at-risk aggregator are preserved through the checkpointing mechanism described in Section 5.2, ensuring no aggregation progress is lost during migration. If no healthy Sub-Global Aggregator instance is available, the Migrator offloads workload to Failover Edge Aggregators, maintaining training continuity during periods of widespread aggregator unavailability.

## 4.2 Aggregators

SpotLight employs two aggregation tiers that share a common implementation, differing only in their position within the hierarchy. Sub-Global Aggregators accumulate updates from clients , while Global Aggregators operate over the sub-global models produced by Sub-Global Aggregator nodes to derive a consistent global model. The following sections describe the aggregator model and state management used in SpotLight.

**Aggregation Model** used in the Sub-Global and Global Aggregators is inspired by FedBuff [20]. The model implemented in SpotLight uses the buffer as a floor for the number of updates required to trigger the aggregation process. This avoids fixing the buffer size and blocking aggregation when the buffer is not full. SpotLight supports five major aggregation algorithms and its modular design allows additional algorithms to be integrated. The supported algorithms are listed with their update equations in Table 2.

**Table 2: Supported Aggregation Algorithms**

| Algorithm | Global Model Update |
|---|---|
| FedAvg [19] | $w^{(t+1)} = \sum_{k=1}^{K} \frac{n_k}{N} w_k^{(t)}$ |
| FedBuff [20] | $w^{(t+1)} = \frac{\sum_{k \in \mathcal{B}} n_k \, w_k^{(t)}}{\sum_{k \in \mathcal{B}} n_k}$ |
| FedNova [30] | $m^{(t+1)} = \beta_m m^{(t)} + (1-\beta_m) \frac{\sum_k p_k (w^{(t)} - w_k^{(t)})/\tau_k}{\sum_k p_k/\tau_k}, \quad w^{(t+1)} = w^{(t)} - \eta_s m^{(t+1)}$ |
| FedAdam [24] | $w^{(t+1)} = w^{(t)} - \eta \cdot \frac{\hat{m}^{(t)}}{\sqrt{\hat{v}^{(t)}}+\varepsilon}, \quad g = w^{(t)} - \bar{w}$ |
| FedYogi [24] | $w^{(t+1)} = w^{(t)} - \eta \cdot \frac{\hat{m}^{(t)}}{\sqrt{\hat{v}^{(t)}}+\varepsilon}, \quad g = \bar{w} - w^{(t)}$ |

**State Management** is supported uniformly across both aggregation tiers. Aggregation algorithms are classified as either stateless, retaining no prior aggregation state, or stateful, maintaining optimizer state. The aggregators persist the global model weights and additionally persist optimizer state for stateful algorithms such as FedAdam, FedNova, and FedYogi. This ensures that an aggregator can be fully restored from its last checkpoint without any loss of state, enabling the Migrator to seamlessly redirect workload to a healthy instance without discarding accumulated aggregation progress.

**Failover Edge Aggregators** are designated edge nodes that assume the Sub-Global or Global Aggregator role during periods of aggregator unavailability caused by provisioning delays or instance boot time. The RM, upon detecting aggregator unavailability, deploys the corresponding aggregator instances onto a statically configured number of Failover Edge Aggregators from the existing pool. Each Failover Edge Aggregator synchronizes itself to the latest model checkpoint from MongoDB before resuming aggregation for the affected tier. The RM treats Failover Edge Aggregators identically to the aggregator tier they replace: they accumulate updates, produce aggregated models, and forward them up the aggregation hierarchy in the same manner. All aggregation state is continuously persisted to MongoDB, ensuring that deactivation is lossless; any updates that cannot be delivered are retained by the RM and redelivered to the next available aggregator instance. Once healthy aggregator instances become available, the RM redirects traffic back to the cloud aggregators and the Failover Edge Aggregator is deactivated.

## 4.3 Update Lifecycle

Client connections to the Resource Manager are maintained as persistent gRPC streams, allowing the RM to manage client state across the full duration of a training session without reconnection overhead. Envoy fronts all inbound client connections, acting as a gateway that manages connection routing to RM instances. The RM forwards client updates to the appropriate Sub-Global Aggregator asynchronously in the background, decoupling client-facing latency from aggregator throughput. The Sub-Global Aggregator accumulates updates in an in-memory buffer and triggers aggregation once the floor threshold is met. The resulting sub-global model is pushed to the Global Aggregator, which incorporates it into the global model. The Global Aggregator does not push the updated global model downstream; instead, Sub-Global Aggregators poll the Global Aggregator for the latest available model. Clients similarly poll the RM for model updates, at which point the RM performs a live pull from the assigned Sub-Global Aggregator and forwards the model to the client.The RM tracks the delivery status of each

forwarded update; a client update is considered delivered only upon receiving an explicit acknowledgment from the target Sub-Global Aggregator. Updates that do not receive an acknowledgment before preemption are retained by the RM and redelivered to the next available aggregator selected by the Scheduler, ensuring no client updates are lost due to aggregator failure.

# 5 Implementation

## 5.1 SpotLight Implementation

SpotLight is implemented in Python across 19.2k lines of code, reflecting the breadth of its control plane, fault tolerance, and scheduling components, and is deployed on a Kubernetes cluster. SpotLight utilizes third-party libraries including Motor, an asynchronous MongoDB driver, gRPC for inter-component communication, NumPy for algorithm implementations, and the Kubernetes Python client for cluster management. SpotLight uses MongoDB for coordination, state management, and client lifecycle tracking. System components are deployed as StatefulSets, which provide stable network identities necessary for gRPC-based inter-component communication, and communicate exclusively over gRPC, chosen over REST for its HTTP/2-native multiplexing and bidirectional streaming, which eliminates per-request connection overhead on the critical path of persistent client-to-RM streams. All inbound client connections are routed through an Envoy gateway, selected for its native gRPC support and connection load balancing. SpotLight was hosted on Chameleon Cloud [13] during evaluation

## 5.2 Model State Management

SpotLight uses MongoDB as its shared persistence layer, storing coordination state accessible across all RM replicas and aggregator instances. Under normal operation, client updates and global model state are managed in-memory across the Sub-Global and Global aggregation tiers. Upon elevated failure likelihood, signaled by the Kaplan-Meier failure estimater (Section **??**), the aggregator persists its in-memory buffer of client updates, current model weights, and server-side optimizer state to MongoDB. For stateful algorithms such as FedAdam [24], server-side optimizer state is additionally persisted; stateless algorithms such as FedAvg [19] require only the model weights. While the aggregator persists its accumulated state, the RM independently tracks the delivery status of all in-flight client updates. As described in Section 4, the RM redelivers any unacknowledged updates to the next available aggregator selected by the Scheduler. All aggregators, whether newly provisioned or already running, maintain a synchronized view of training state by continuously pulling the latest model parameters and state from MongoDB. This design ensures no client updates are lost due to preemptions or transient network failures.

## 5.3 Aggregator Failover

Spot VM provisioning introduces non-trivial startup latency that can exceed the preemption warning window provided by cloud providers [7, 31], during which training would otherwise stall. SpotLight bridges this window using a statically configured pool of Failover Edge Aggregators that assume the role of the unavailable cloud aggregator for the duration of the provisioning delay. The RM polls aggregator health at 0.1 second intervals, minimizing the latency between detecting unavailability and activating a Failover Edge Aggregator. The RM determines which aggregator tier has become unavailable and assigns the corresponding Sub-Global or Global Aggregator role to the Failover Edge Aggregator. Failover Edge Aggregators pull the latest model parameters and state from MongoDB before accepting client updates, consistent with the persistence model described in Section 5.2. Once the newly provisioned cloud aggregator is detected as ready, the RM redirects traffic back and deactivates the edge instance.

## 5.4 Spot Emulator

A key feature of spot instances is their variability in preemption behavior, driven by factors including region, availability zone, instance type, temporal market conditions and cloud providers [4, 16, 26]. This variability across configurations makes real-world evaluation of application behavior on spot instances difficult to reproduce consistently and cost-prohibitive at scale, as comprehensive coverage across regions and instance types requires extensive testing for long periods of time. Therefore, to evaluate SpotLight under realistic spot conditions, we developed the Spot Emulator, a high-fidelity trace-driven emulator that replays spot VM behavior using publicly available spot availability traces [14, 32]. The emulator is designed around two key goals. First, high fidelity: the emulator replays preemption events derived from real-world availability traces, ensuring emulated behavior reflects realistic spot market conditions. Second, controller-agnostic design: the modular architecture of the Spot Emulator allows it to support Spot VM behavior replay with different cloud services providers and deploy on non-Kubernetes platforms with minimal changes to the emulator

The lifetime distribution used to replay spot behavior is derived from the availability traces in [32]. The availability trace is used to derive a CDF, which allows the emulator to reliably replay spot instance lifetime dynamics. To ensure that evaluation cost is realistic, price traces obtained from [14] are used to generate realistic execution costs. As shown in Figure 5, survival probability decreases monotonically with instance lifetime, reflecting the inherent unreliability of spot instances over time.

The Spot Emulator consists of five components. The first is the *Distribution Extractor*, which filters the availability trace by region, availability zone, and OS, and derives the lifetime CDF from the filtered data. The second is the *Lifecycle Manager*, which samples a lifetime for each newly provisioned VM from the CDF within the P10–P90 range, excluding tail outliers that represent atypically short or long-lived instances, and maintains an internal clock tracking the lifetime of all active spot VMs. Once a VM's assigned lifetime expires, the Lifecycle Manager triggers the third component, the *Executor*, which terminates the instance via the Kubernetes API. The fourth component is the *Control Plane*, which manages the execution order of the other components and exposes configuration parameters to the user. The fifth component is the *Cost Report Generator*, which uses price traces from [14] to produce accurate execution cost reports. In our evaluation, the Control Plane was configured to sample lifetimes between P10 and P90 and to apply a 100× time compression factor, allowing the full range of the lifetime distribution to be exercised within the evaluation window.

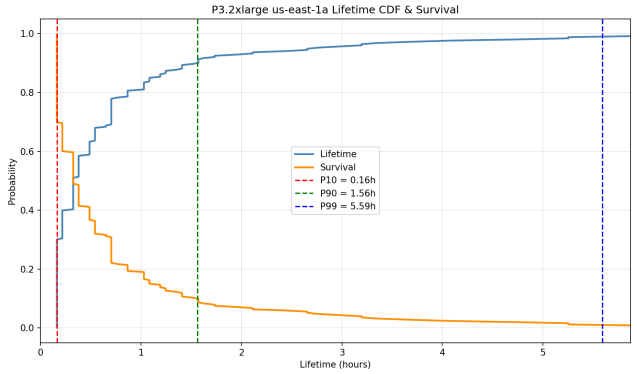



**Figure 5: Lifetime CDF and survival function $S(t)$ for `p3.2xlarge` instances in `us-east-1a`, derived from availability traces in [32]. The distribution is heavy-tailed: 10% of instances are preempted within 0.16 hours (P10), 90% within 1.56 hours (P90), and 99% within 5.59 hours (P99). The Spot Emulator samples instance lifetimes from this CDF to replay preemption events during evaluation.**

The Kaplan-Meier estimator [12] is a non-parametric survival model that estimates the probability of survival of a population over time. The Kaplan-Meier estimator runs in the background alongside the Spot Emulator, continuously estimating the survival probability of active spot VM instances based on their observed age and publishing per-instance survival probability signals to MongoDB. The Migrator and aggregators utilize them to proactively redirect workload away from instances at elevated risk of preemption and checkpoint in-memory model state before preemption occurs, as described in Section 4.1 and Section 5.2.

The emulator is deployed on a physical server that is logically connected to virtual machines through Kubernetes. The physical machine serves as the control plane and the worker nodes as servers in a datacenter. The pods deployed on the workers nodes are treated by the emulator as Spot VMs and are terminated based on their assigned lifetime.

## 6 Evaluation Setup

SpotLight was evaluated on Chameleon Cloud [13], a platform designed for experimental systems research. SpotLight was deployed on a five-node Kubernetes cluster, with node specifications listed in Table 3. MongoDB was hosted on a dedicated machine outside the cluster, with equivalent specifications to the host node. The evaluation focuses on two key aspects of the system: scalability and fault tolerance.

**Table 3: Cluster Node Specifications**

| Specification | Host | Worker |
|---|---|---|
| CPU Model | Intel Xeon Gold 6126 @ 2.60GHz | — |
| vCPUs | 48 | 16 |
| Total RAM | 187 GiB | 28 GiB |

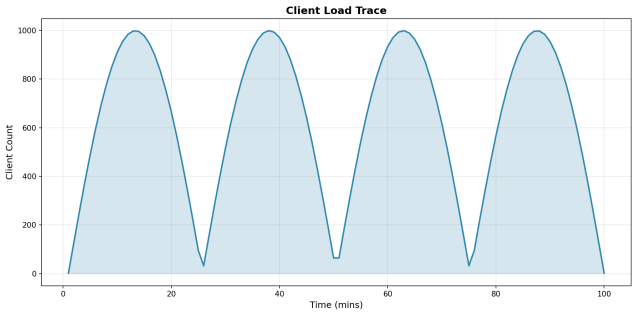



**Figure 6: Cyclical workload pattern used in the scalability evaluation, reflecting natural increases and decreases in active client participation throughout the day.**

### 6.1 Scalability

The scalability of SpotLight is evaluated across four metrics that capture the impact of increasing client load on system performance. SpotLight is evaluated against Papaya [8], a production asynchronous FL system, as the baseline, chosen for its support of asynchronous aggregation at scale. The workload used for the evaluation represents a cyclical pattern of client participation, reflecting natural increases and decreases in active client populations throughout the day, as shown in Figure 6. The evaluation uses a CNN model with a 10MB payload to measure system scalability. The metrics used are as follows.

**1) Client Utilization.** Client utilization is defined as $\frac{T_{train}}{T_{train}+T_{aggregation}+T_{queue}}$, where $T_{train}$ denotes local computation at the client, $T_{aggregation}$ denotes the time spent waiting for aggregation to complete, and $T_{queue}$ denotes the time buffered at the Sub-Global Aggregator prior to aggregation. Higher utilization indicates a greater fraction of total time spent on useful local computation rather than coordination overhead.

**2) Aggregator Scaling.** This metric evaluates how the aggregation tier scales relative to client load across all supported algorithms. A scalable design should demonstrate proportional and controlled growth in Sub-Global Aggregator instances as client population increases.

**3) Queue Time.** Queue time measures the duration each client spends waiting before its update enters aggregation, capturing contention at the aggregation layer. In a well-scaled system, queue time should remain bounded as client count grows.

**4) Cost.** Cost measures the total infrastructure expenditure required to operate SpotLight under varying client loads, expressed as instance-hours consumed by the aggregation tier over the duration of training.

Since Papaya does not support elastic scaling in response to client load, two configurations were derived to enable a fair comparison. SpotLight was first run on the workload shown in Figure 6, and its execution cost and average client utilization were recorded. These two parameters were used to derive the following Papaya configurations:

**Papaya-Budget** deploys a fixed number of aggregators at the same cost as SpotLight, measuring the client utilization and queue time achievable within an equivalent budget.

**Papaya-Max** determines the additional expenditure required for Papaya to match SpotLight's client utilization and queue time, quantifying the cost overhead of operating without elastic scaling.

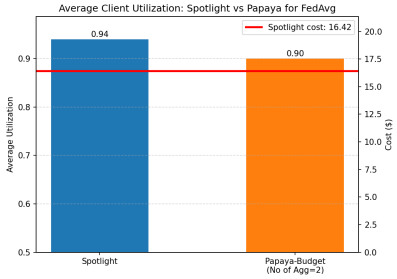
The evaluation were conducted for all algorithms listed in Table 2. The results for SpotLight vs Papaya-Budget are shown in Figure 7.

# 7 Discussion

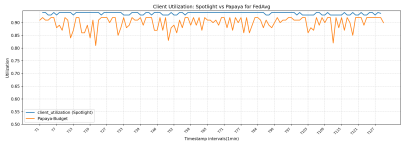
# 8 Related Work

# 9 Conclusion

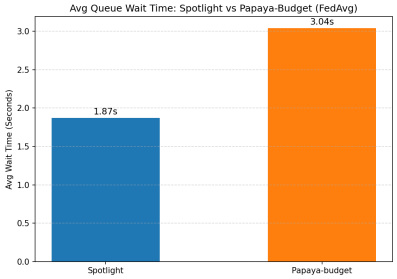
We present SpotLight...




(a) Average client utilization: SpotLight achieves 0.94 vs Papaya-Budget at 0.90 within the same cost budget ($16.42).



(b) Client utilization over time: SpotLight maintains stable utilization while Papaya-Budget exhibits variance due to fixed aggregator capacity.




(c) Average queue wait time: SpotLight reduces wait time to 1.87s compared to 3.04s for Papaya-Budget, a 38.5% reduction.

**Figure 7: Papaya-Budget comparison for FedAvg: SpotLight achieves higher client utilization and lower queue wait time than Papaya-Budget at equivalent cost.**

# References


[1] Amazon Web Services. 2024. EC2 Spot Instance Interruption Notices. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-instance-termination-notices.html. Accessed: 2024.

[2] Daniel J. Beutel, Taner Topal, Akhil Mathur, Xinchi Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Kwing Hei Li, Titouan Parcollet, Pedro Porto Buarque de Gusmão, and Nicholas D. Lane. 2022. Flower: A Friendly Federated Learning Research Framework. *arXiv preprint arXiv:2007.14390* (2022).

[3] Kallista A. Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloé Kiddon, Jakub Konecný, Stefano Mazzocchi, H. Brendan McMahan, Timon Van Overveldt, David Petrou, Daniel Ramage, and Jason Roselander. 2019. Towards Federated Learning at Scale: System Design. In *SysML*.

[4] Jiangfei Duan, Ziang Song, Xupeng Miao, Xiaoli Xi, Dahua Lin, Harry Xu, Minjia Zhang, and Zhihao Jia. 2024. Parcae: proactive, liveput-optimized DNN training on preemptible instances. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation* (Santa Clara, CA, USA) *(NSDI'24)*. USENIX Association, USA, Article 62, 19 pages.

[5] Kazim Ergun, Rishikanth Chandrasekaran, and Tajana Rosing. 2023. Federated Hyperdimensional Computing. *arXiv preprint arXiv:2312.15966* (2023).

[6] Google Cloud. 2024. Spot VMs. https://cloud.google.com/compute/docs/instances/spot. Accessed: 2024.

[7] Jianwei Hao, Ting Jiang, Wei Wang, and In Kee Kim. 2021. An Empirical Analysis of VM Startup Times in Public IaaS Clouds. In *2021 IEEE 14th International Conference on Cloud Computing (CLOUD)*. 398–403. doi:10.1109/CLOUD53861.2021.00053

[8] Dzmitry Huba, John Nguyen, Kshitiz Malik, Ruiyu Zhu, Mike Rabbat, Ashkan Yousefpour, Carole-Jean Wu, Hongyuan Zhan, Pavel Ustinov, Harish Srinivas, Kaikai Wang, Anthony Shoumikhin, Jesik Min, and Mani Malek. 2022. PAPAYA: Practical, Private, and Scalable Federated Learning. In *Proceedings of the 5th Conference on Machine Learning and Systems (MLSys 2022)*. https://proceedings.mlsys.org/paper_files/paper/2022/hash/a8bc4cb14a20f20d1f96188bd61eec87-Abstract.html

[9] David Irwin, Prashant J. Shenoy, Pradeep Ambati, Prateek Sharma, Supreeth Shastri, and Ahmed Ali-Eldin. 2019. The Price Is (Not) Right: Reflections on Pricing for Transient Cloud Servers. In *28th International Conference on Computer Communication and Networks (ICCCN)*. 1–9.

[10] K. R. Jayaram, Vinod Muthusamy, Gegi Thomas, Ashish Verma, and Mark Purcell. 2022. Adaptive Aggregation for Federated Learning. *arXiv preprint arXiv:2203.12163* (2022).

[11] Pedro Joaquim, Manuel Bravo, Luís Rodrigues, and Miguel Matos. 2019. Hourglass: Leveraging Transient Resources for Time-Constrained Graph Processing in the Cloud. In *The Fourteenth EuroSys Conference (EuroSys)*.

[12] E. L. Kaplan and Paul Meier. 1958. Nonparametric Estimation from Incomplete Observations. *J. Amer. Statist. Assoc.* 53, 282 (1958), 457–481. doi:10.1080/01621459.1958.10501452

[13] Kate Keahey, Jason Anderson, Zhuo Zhen, Pierre Riteau, Paul Ruth, Dan Stanzione, Mert Cevik, Jacob Colleran, Haryadi S. Gunawi, Cody Hammock, Joe Mambretti, Alexander Barnes, François Halbach, Alex Rocha, and Joe Stubbs. 2020. Lessons Learned from the Chameleon Testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association.

[14] Sungjae Lee, Jaeil Hwang, and Kyungyong Lee. 2022. SpotLake: Diverse Spot Instance Dataset Archive Service. In *2022 IEEE International Symposium on Workload Characterization (IISWC)*. 242–255. doi:10.1109/IISWC55918.2022.00029

[15] Mu Li, David G. Andersen, Jun Woo Park, Alexander J. Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J. Shekita, and Bor-Yiing Su. 2014. Scaling Distributed Machine Learning with the Parameter Server. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, Broomfield, CO, 583–598.

[16] Zhifei Li, Tian Xia, Ziming Mao, Zihan Zhou, Ethan J. Jackson, Jamison Kerney, Zhanghao Wu, Pratik Mishra, Yi Xu, Yifan Qiao, Scott Shenker, and Ion Stoica. 2026. SkyNomad: On Using Multi-Region Spot Instances to Minimize AI Batch Job Cost. *CoRR* abs/2601.06520 (2026). arXiv:2601.06520 doi:10.48550/arXiv.2601.06520

[17] Yang Liu, Tao Fan, Tianjian Chen, Qian Xu, and Qiang Yang. 2021. FATE: An Industrial Grade Platform for Collaborative Learning with Data Protection. *Journal of Machine Learning Research* 22, 226 (2021), 1–6.

[18] Heiko Ludwig, Nathalie Baracaldo, Gegi Thomas, Yi Zhou, Ali Anwar, Shashank Rajamoni, Yuya Ong, Jayaram Radhakrishnan, Ashish Verma, Mathieu Sinn, et al. 2020. IBM Federated Learning: An Enterprise Framework White Paper V0.1. *arXiv preprint arXiv:2007.10987* (2020).

[19] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*. 1273–1282.

[20] John Nguyen, Kshitiz Malik, Hongyuan Zhan, Ashkan Yousefpour, Mike Rabbat, Mani Malek, and Dzmitry Huba. 2022. Federated Learning with Buffered Asynchronous Aggregation. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 3581–3607.

[21] NVIDIA. 2025. Efficient Federated Learning in the Era of LLMs with Message Quantization and Streaming. NVIDIA Technical Blog. https://developer.nvidia.com/blog/efficient-federated-learning-in-the-era-of-llms-with-message-quantization-and-streaming/

[22] Diego Ongaro and John Ousterhout. 2014. In Search of an Understandable Consensus Algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)*. USENIX Association, Philadelphia, PA, 305–319. https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro

[23] Shixiong Qi, K. K. Ramakrishnan, and Myungjin Lee. 2024. LIFL: A Lightweight, Event-driven Serverless Platform for Federated Learning. In *Proceedings of Machine Learning and Systems (MLSys)*.

[24] Sashank Reddi, Zachary Charles, Manzil Zaheer, Zachary Garrett, Keith Rush, Jakub Konečný, Sanjiv Kumar, and H. Brendan McMahan. 2021. Adaptive Federated Optimization. In *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2003.00295

[25] Ruitao Shang, Fei Xu, Zhuoyan Bai, Li Chen, Zhi Zhou, and Fangming Liu. 2023. spotDNN: Provisioning Spot Instances for Predictable Distributed DNN Training in the Cloud. In *2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS)*. 1–10.

[26] Prateek Sharma, JCS Kadupitiya, and Vikram Jadhao. 2019. Modeling Constrained Preemption Dynamics Of Transient Cloud Servers. *arXiv* (2019), arXiv–1911.

[27] Supreeth Subramanya, Amr Rizk, and David Irwin. 2016. Cloud Spot Markets are Not Sustainable: The Case for Transient Guarantees. In *8th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*.

[28] Yan Sun, Li Shen, Hao Sun, Liang Ding, and Dacheng Tao. 2023. Efficient Federated Learning Via Local Adaptive Amended Optimizer With Linear Speedup. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 12 (2023), 14453–14464. doi:10.1109/TPAMI.2023.3300886

[29] John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. 2023. Bamboo: Making Preemptible Instances Resilient for Affordable Training of Large DNNs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 497–513.

[30] Jianyu Wang, Qinghua Liu, Hao Liang, Gauri Joshi, and H. Vincent Poor. 2020. Tackling the Objective Inconsistency Problem in Heterogeneous Federated Optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2007.07481

[31] Zhanghao Wu et al. 2024. SkyServe: Serving AI Models across Regions and Clouds with Spot Instances. *arXiv preprint arXiv:2411.01438* (2024).

[32] Zhanghao Wu, Wei-Lin Chiang, Ziming Mao, Zongheng Yang, Eric Friedman, Scott Shenker, and Ion Stoica. 2024. Can't Be Late: Optimizing Spot Instance Savings under Deadlines. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 185–203.

[33] Cong Xie, Sanmi Koyejo, and Indranil Gupta. 2019. Asynchronous Federated Optimization. *arXiv preprint arXiv:1903.03934* (2019).

[34] Fangkai Yang, Lu Wang, Zhenyu Xu, Jue Zhang, Liqun Li, Bo Qiao, Camille Couturier, Chetan Bansal, Soumya Ram, Si Qin, Zhen Ma, Íñigo Goiri, Eli Cortez, Terry Yang, Victor Rühle, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. 2023. Snape: Reliable and Low-Cost Computing with Mixture of Spot and On-Demand VMs. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.